

MONTH 1 LEARNING PLAN

UI, Modern CSS & React Foundations

A visual study guide for building, reviewing, and debugging with confidence.

Build

Semantic HTML, responsive layouts, data demos, and a React task board.

Understand

Box model, Flexbox, immutable data, asynchronous requests, state, and data flow.

Validate

Use DevTools, precise error reports, code review checklists, and independent diagnosis.

Your job with AI-assisted software is to review, validate, and debug what was generated.

AT A GLANCE

Four-Week Roadmap

WEEK 1 Modern Layouts Master the box model, Flexbox axes, responsive constraints, and DevTools inspection. 01 Box model and border-box 02 Flexbox alignment 03 Responsive widths 04 Inspect winning CSS rules 05 Build a two-column page	WEEK 2 Data Flow Transform data without mutation and handle asynchronous requests visibly. 01 Values and references 02 map , filter , and spread 03 fetch lifecycle 04 Network debugging 05 Build a data demo	WEEK 3 React Architecture Build predictable components with props, state, keys, and client boundaries. 01 Component hierarchy 02 State and events 03 Immutable collections 04 Derived state and keys 05 Build a task board	WEEK 4 Review & Integration Prompt precisely, diagnose independently, review AI output, and teach back the system. 01 Prompt architecture 02 Code review drill 03 Debugging drill 04 Refactor components 05 Capstone and teach-back
--	--	---	--

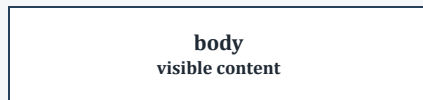
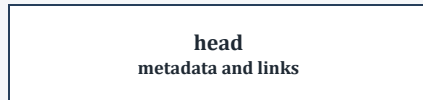
Daily Session Pattern



One small sandbox project should grow throughout the month.

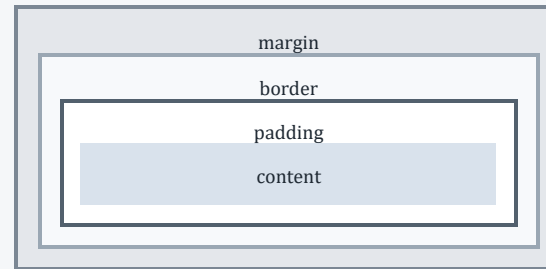
How the Core Ideas Fit Together

HTML structure



```
<header> <main> <footer>
```

CSS box model



border-box includes padding and borders inside the declared size.

```
* { box-sizing: border-box; }
```

Flexbox axes

cross axis



main axis ->

- **justify-content** follows the main axis.
- **align-items** follows the cross axis.
- **flex-direction** chooses the main axis.

JavaScript transformation



React data flow



```
setCount(prev => prev + 1)
```

Immutable update

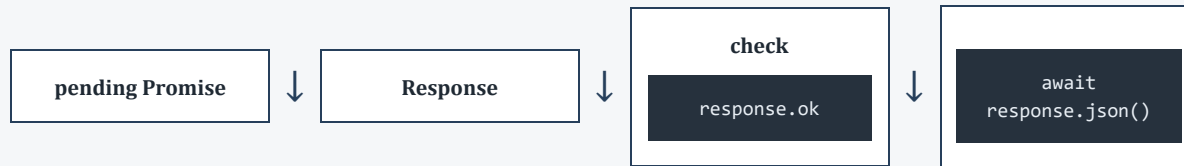
References matter. Create a new outer array or object so React can detect the change.

```
setItems([...items, newItem])  
  
items.map(update)  
items.filter(keep)
```

Remember: derived values should be calculated from existing state instead of stored redundantly.

Trace the Request, Then Explain the Failure

Fetch lifecycle



Important: HTTP 400 and 500 responses usually do not reject

```
fetch
; check

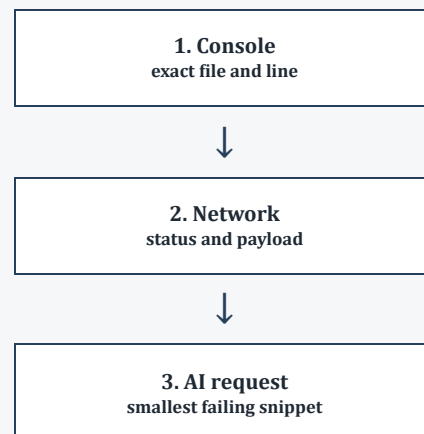
response.ok

yourself.
```

Network inspection

- **Headers:** URL, method, and status.
- **Payload:** JSON sent by the client.
- **Response:** raw JSON returned by the server.
- **States:** loading, success, empty, and error.

Systematic error triage



AI review checklist

[]

```
use client
```

when hooks or events are used.

[] State updates are immutable.

[] List keys are stable and unique.

[] Layouts use responsive constraints.

[] Derived values are not redundant state.

Make one independent diagnosis before requesting generated code.

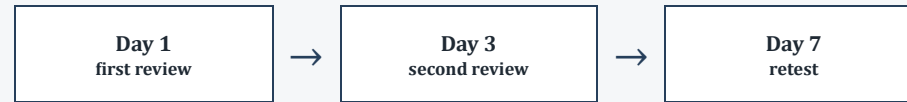
Record the exact error, expected behavior, and smallest relevant snippet.

Evidence, Review, and Month-End Check

Daily evidence

- ☐ One code sample.
- ☐ One screenshot or console/network observation.
- ☐ One sentence explaining the lesson.
- ☐ One independent diagnosis before asking AI.

Spaced review



Mark a term learned only after defining and applying it correctly twice without notes.

Portfolio evidence

- ☐ Week 1 responsive layout.
- ☐ Week 2 data demo.
- ☐ Week 3 React task board.
- ☐ Week 4 reviewed capstone.

Month 1 Self-Assessment

- ☐ Explain why

```
border-box
```

matters.

- ☐ Build a responsive two-column layout.
- ☐ Explain mutation versus shallow copy.

- ☐ Explain why

```
push()
```

can prevent a React update.

- ☐ Diagnose a failed network request.
- ☐ Resolve a missing

```
use client
```

boundary.